

Context-Free Languages

① Context-Free Grammar (CFG)

Definition: A CFG is a set of rules / productions used to generate patterns of strings.

* $L = \{ \text{string containing 0 and 1} \}$ $L = \{ 0^n 1^n \mid n \geq 1 \}$

sols if $n=1$; $0^1 1^1 = 01$

" $n=2$; $0^2 1^2 = 0011$

" $n=3$; $0^3 1^3 = 000111$

CFG:

$A \rightarrow 01 \mid 0A1$

or. $A \rightarrow 01$

$A \rightarrow 0A1$

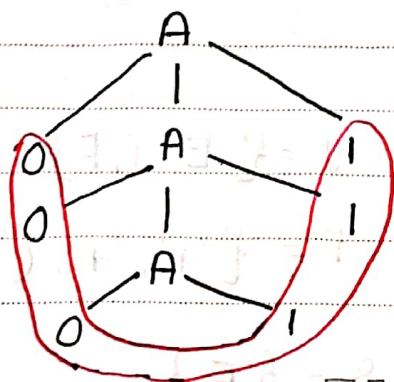
* generate pattern $\rightarrow 000111$

$\therefore A \rightarrow 0A1$

$\rightarrow 00A11$; here $A \rightarrow 0A1$

$\rightarrow 000111$; here $A \rightarrow 01$

Parsing :-



parse tree. $\therefore$ parse tree
is or string valid
otherwise not.

* A context free grammar is a 4-tuple (V, T, R, S)
 $\begin{matrix} V \\ \underline{N} \end{matrix}, \begin{matrix} T \\ \underline{\Sigma} \end{matrix}, \begin{matrix} R \\ \underline{P} \end{matrix}, S$

① $V \rightarrow$ variable, $N \rightarrow$ Non-terminal.

② $T \rightarrow$ Terminal, $\Sigma \rightarrow$ alphabet.

③ $R \rightarrow$ Rules, $P \rightarrow$ Productions.

④ $S \rightarrow$ starting variable.

$$S \in V$$

$\{ \in = \text{belongs to} \}$

ex: CFG

L.H.S
variable

$$\begin{array}{l} A \rightarrow OA1 \\ A \rightarrow B \\ B \rightarrow \# \end{array}$$

R.H.S

terminal

substitution rule $\rightarrow$ process is called derivation.

starting variable $(S) \rightarrow A$. यदि जो लउर ना शक गइल
 गइल ओर शक अ S. Page: 103, Figure 2.1.

Identify the terminals, nonterminals, start variable for given CFG:

$$\begin{array}{l} ① E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid a \end{array}$$

sol:

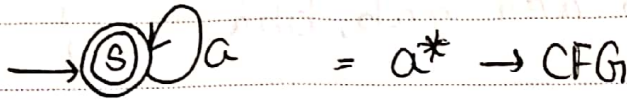
$$N = \{ E, T, F \}$$

$$T = \{ '+', '*', '(', ')', 'a' \}$$

$$S = \{ E \}$$

① $L = \{a^n \mid n \geq 0\} = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

sol:



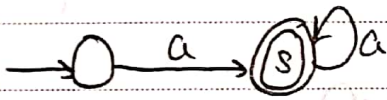
$= a^* \rightarrow \text{CFG}$

$\{a^* = \text{any no. of } a \text{ read}\}$
 $\{n \text{ any no. of } \epsilon\}$

$S \rightarrow aS \mid \epsilon$

② $L = \{a^n \mid n \geq 1\} = \{a, aa, aaa, \dots\}$

sol:



$a^+ \rightarrow \text{CFG}$

$S \rightarrow aS \mid a$

or,

$S \rightarrow aS$

$S \rightarrow a$

$\{a^+ = \text{at least one } a \text{ read}\}$

③ $L = \{\text{set of all strings over } a \text{ and } b\}$
 $= \{\epsilon, a, b, aa, ab, bb, ba, \dots\}$

sol:



$\{(a+b)^* = \epsilon \text{ any no.}\}$

or, $(a \cup b)^*$

or, $(a+b)^*$

or, $(a|b)^*$

we can write any one but we choose this.

$\text{CFG} \Rightarrow S \rightarrow aS \mid bS \mid \epsilon$

① Construct CFG for set of all strings which length at least 2. $\geq(a,b)$.

sol: $\{aa, ab, ba, bb, aaa, aab, bba, \dots\}$

$$\frac{(a+b)}{A} \quad \frac{(a+b)}{A} \quad \frac{(a+b)^*}{B}$$

CFG $\Rightarrow S \rightarrow AAB$

$A \rightarrow a|b$

$B \rightarrow aB|bB|\epsilon$

$\rightarrow \text{Diagram of } a, b$

② Construct CFG for set of all strings at least three 0's.

sol: $\{1010101, 0000, 100101, 0000, \dots\}$

$S \rightarrow AOA0A0A$

$A \rightarrow (0+1)^*$

$A \rightarrow 0A|1A|\epsilon$

$\rightarrow \text{Diagram of } 0, 1$

③ Construct CFG for set of all strings which at most $\leq (a, b)$.

sol: $\{\epsilon, a, b, aa, bb, ba, ab\}$.

$$\frac{(a+b+\epsilon)}{A} \quad \frac{(a+b+\epsilon)}{A}$$

$$S \rightarrow AA$$

$$A \rightarrow a|b|\epsilon$$

④ Construct CFG for ^{set of} all strings starts with a and ends with b.

sol: $\{ab, aab, abb, \dots\}$.

$$\therefore a \frac{(a+b)^*}{A} b$$

$$S \rightarrow aAb$$

$$A \rightarrow aA|bA|\epsilon \rightarrow \textcircled{A}^{a,b}$$

⑤ Construct CFG start and end with different symbol.

sol: $\{ab, ba, aab, bba, \dots\}$

$$\therefore a(a+b)^*b$$

$$\text{or } b(a+b)^*a$$

$$S \rightarrow aAb|bAa$$

$$A \rightarrow aA|bA|\epsilon$$

Ambiguity:

□ If a grammar generates the same string in several different ways, we say that the string is derived ambiguously in that grammar. If a grammar generates some string ambiguously, we can say that the grammar is ambiguous.

CFG:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned} \left. \vphantom{\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}} \right\} \text{ambiguous grammar}$$

* Draw pattern / string $\rightarrow a + a * a$ Ambiguously generated string.

$$\begin{aligned} E &\rightarrow E + T \\ &\rightarrow T + T \times F \\ &\rightarrow F + F \times a \\ &\rightarrow a + a \times a \end{aligned}$$

Remove Null production from CFGs

☐ Directly ϵ or null produce means CFG আর variable ϵ or directly ϵ produce করে।

ex: $S \rightarrow AB$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$ } কারণ S এর A and B ϵ হতে পারে।
 $A \rightarrow \epsilon$ and $B \rightarrow \epsilon$.

No(X) null production

ex: $S \rightarrow ABC$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$
 $C \rightarrow c$ } কারণ S NP রপে
 and S এর C
 directly NP হতে পারে।

ex: Remove NP given CFG.

$S \rightarrow ABC$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$
 $C \rightarrow c$

Removing $A \rightarrow \epsilon$

$S \rightarrow \underline{A}BC | BC$
 $A \rightarrow a\underline{A} | a$
 $B \rightarrow bB | \epsilon$
 $C \rightarrow c$

Again, removing $B \rightarrow \epsilon$

$S \rightarrow A\underline{B}C | \underline{B}c | AC | c$
 $A \rightarrow aA | a$
 $B \rightarrow b\underline{B} | b$
 $C \rightarrow c$

* এই production a grammar aও তার কোন effect নেই।

{ unreachable error
 CFG is also
 variable error not
 also error in the form

Chomsky Normal form

ଯଦି ଗୋଟିଏ ଗ୍ରାମର variable ଗୋଟିଏ terminal କିମ୍ବା ଗୋଟିଏ variable କିମ୍ବା concatenation produce କରେ।

$$A \rightarrow BC$$

$$A \rightarrow a$$

R.H.S

$a \rightarrow$ terminal

$BC \rightarrow$ concatenation, $B, C \in V$.

ex: $S \rightarrow ASA | aB$

$$A \rightarrow B | s$$

$$B \rightarrow b | \epsilon$$

convert CFG to CNF.

Step 1:

ଯଦି starting variable ଥିବା R.H.S appear କରେ ତେବେ ଏହି variable ଫିଟ କର।

$\therefore$ Add ($s_0 \rightarrow S$).

$$s_0 \rightarrow S$$

$$S \rightarrow ASA | aB$$

$$A \rightarrow B | s$$

$$B \rightarrow b | \epsilon$$

Step 2:-

* Remove null production.

$$B \rightarrow \epsilon, A \rightarrow \epsilon$$

Now, Removing $B \rightarrow \epsilon$,

$$S_0 \rightarrow S$$

$$S \rightarrow ASA | \underline{a}B | a$$

$$A \rightarrow \underline{B} | S | \epsilon$$

$$B \rightarrow b$$

Again, Removing $A \rightarrow \epsilon$

$$S_0 \rightarrow S$$

$$S \rightarrow \underline{A}S | \underline{a}B | a | SA | AS | S$$

$$A \rightarrow B | S$$

$$B \rightarrow b$$

Step 3:-

* Remove unit productions

$$S_0 \rightarrow S, S \rightarrow S, A \rightarrow B, A \rightarrow S$$

$$\therefore S_0 \rightarrow ASA | aB | a | SA | AS$$

$$S \rightarrow ASA | aB | a | SA | AS$$

$$A \rightarrow b | ASA | aB | a | SA | AS$$

$$B \rightarrow b$$

Step 4:-

* For concatenation 2 or more are given 2 or more
2 or more.

for 2nd, $x \rightarrow SA$.

$$S_0 \rightarrow Ax|aB|a|SA|AS$$

$$S \rightarrow Ax|aB|a|SA|AS$$

$$A \rightarrow b|Ax|aB|a|SA|AS$$

$$B \rightarrow b$$

$$X \rightarrow SA$$

step 5:-

* terminal and variable answer and non-terminal

$$\text{let, } Y \rightarrow a$$

$$S_0 \rightarrow Ax|YB|a|SA|AS$$

$$S \rightarrow Ax|YB|a|SA|AS$$

$$A \rightarrow b|Ax|YB|a|SA|AS$$

$$B \rightarrow b$$

$$X \rightarrow SA$$

$$Y \rightarrow a.$$

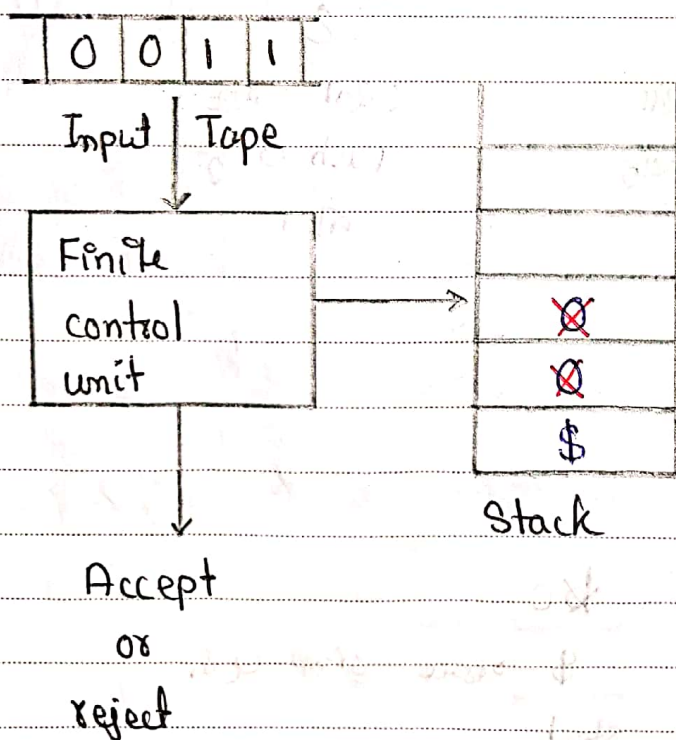
② Push down Automata (PDA)

□ PDA એક machine or computational model

language accept કરે or Reject કરે.

અંતિ language context free language એ. એક PDA આવી accept કરે. એ NFA એ. બરાબ but extra ભાગો component અને એક stack. $\therefore \text{PDA} = \text{FSM} + \text{stack}$.
state machine

* $L = \{0^n 1^n \mid n \geq 0\}$, here $n=2$. એક figure 2.15 page 115.
 $\therefore \{0011, 0101, \dots\}$



* stack initially એક \$ sign અને.

* અંતિ 0 પર એક push U.

* અંતિ 1 તરફ એક Pop U.

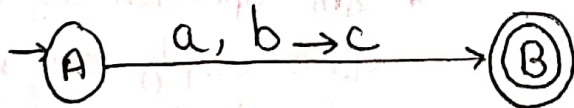
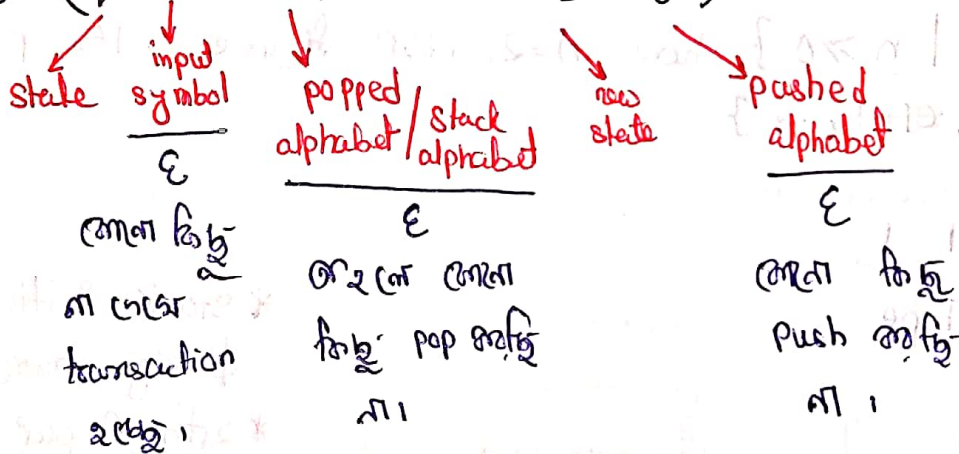
* અંતિ operation એ એક stack empty અને means only \$ sign અને એક $0^n = 1^n$ string accepted. અને last \$ pop અને બરાબ.

* Formal Definition Page 113.

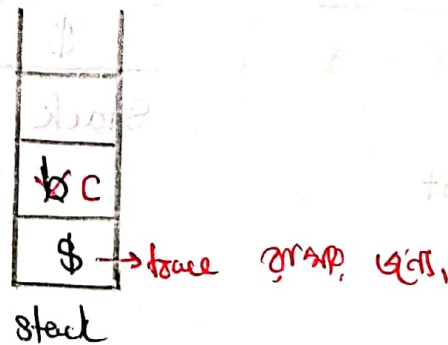
Formal Definitions

- ① Q = Finite set of states.
- ② Σ = Input Alphabet.
- ③ Γ = stack alphabet. *or alphabet stack a maza.*
- ④ $\delta : Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma)$ is the transition function.

another ways $(q, a, x) \rightarrow (q_1, y)$



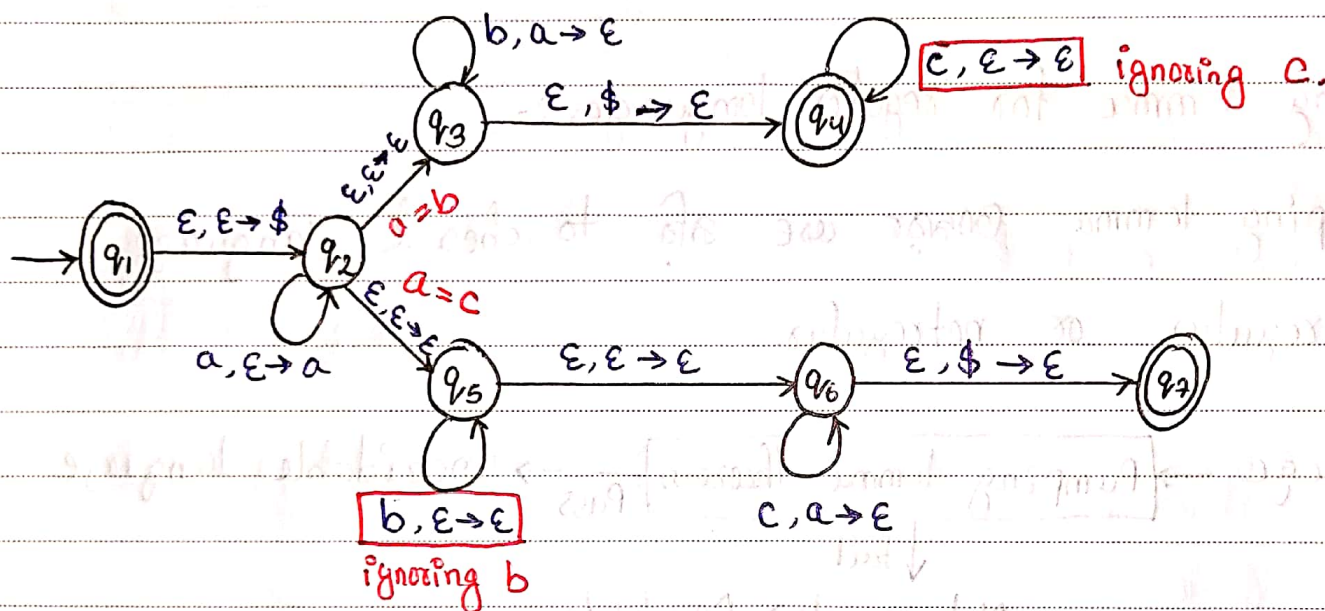
Here, a = input symbol.
 b = popped element.
 c = pushed element.



$\therefore (A, a, B) \rightarrow (B, c)$
 transition function.

- ⑤ $q_0 \in Q$ = start state.
- ⑥ $F \subseteq Q$ = Final state from ex = B .
 from ex = A .

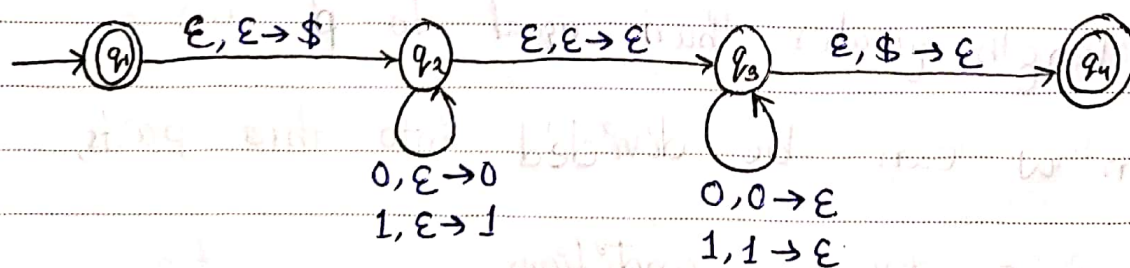
* Draw PDA for, $L = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i=j \text{ or } i=k\}$
 sol: $\{aabbccc, aabbbbcc, \dots\}$ either $a=b$ or $a=c$.



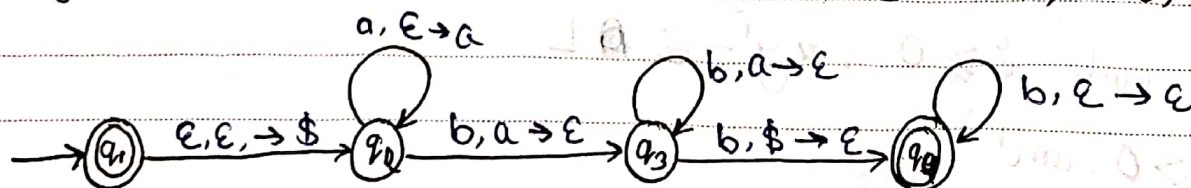
* Draw PDA for, $L = \{ww^R \mid w \in \{0,1\}^*\}$

sol: w = string, w^R = string reverse. $\therefore$ its palindrome.

$L = \{001100, 110011, 0110, \dots\}$



* Design PDA for $L = \{a^m b^n \mid m < n\}$. $\{aobbbb, abbb, \dots\}$



☐ Draw state diagram from given CFG.

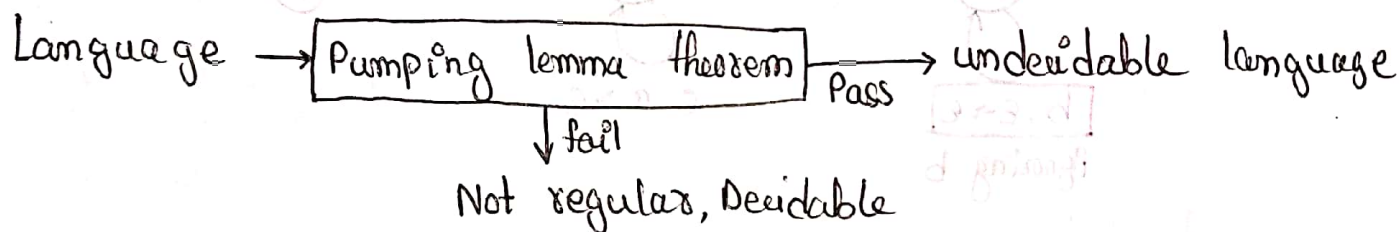
$S \rightarrow aTb|b$

$T \rightarrow Ta| \epsilon$

Sol: Page: 120. Figure 2.26.

Pumping lemma for regular languages:-

☐ Pumping lemma can be used to check language is regular or not regular.



* If L is an infinite language then there exist some positive integer 'p' (pumping length) such that any string $w \in L$ has length greater than equal to 'p' i.e.

$|w| \geq p$ then w can be divided into three parts,

$w = xyz$ satisfy following conditions

① for each $i \geq 0$, $xy^iz \in L$

② $|y| > 0$, and

③ $|xy| \leq p$.

ex: $L = \{a^n b^n \mid n \geq 1\} \therefore L = \{aabb, aaabbb, \dots\}$

→ suppose we took $n=2$ and $w=aabb$.

for pumping lemma: $\frac{a}{x} \frac{a}{y} \frac{b}{z} \frac{b}{z}$

case 3: $|xyz| \leq p$

$$= 3 \leq 4 \checkmark$$

Here, p = total string length

case 2: $|y| > 0$

$$= 2 > 0 \checkmark$$

case 1: $w = x y^i z \in L \quad i \geq 0$

* assume i value error for $w \notin L$.

$$\therefore w = x y^i z = x y^2 z \quad \therefore i=2$$

$$= a(ab)^2 b$$

$$w = aababb$$

y error value pump
error i value
error

Here, $w \notin L$. $\times$

* it means case 1 Pass error.

Example: for case pass error error language/string
is undecidable error but for ~~error~~ fail error
not regular language.

$\therefore L = \text{not regular}$

Pumping lemma for context-free languages:-

to prove certain languages are not context-free.

- condition / way to check PL regular language or not just can't case gram different.

* If L is a context-free language, then there is a number p (the pumping length) where, if w is any string in L of length at least p , $|w| \geq p$. the w divided into five pieces $w = uvxyz$ satisfying the condition.

① for each $i \geq 0$, $uv^i x y^i z \in L$.

② $|y| > 0$, and

③ $|vxy| \leq p$.

ex^o $L = \{a^n b^n c^n \mid n \geq 1\}$, let, $n=2$.

$\therefore L = \{aabbcc\}$.

$\therefore w = \frac{a}{u} \frac{ab}{v} \frac{bc}{x} \frac{c}{y} \frac{c}{z}$

case 38 - $|vxy| \leq p$

$$= 4 \leq 6. \checkmark$$

Case 2 - $|v y| > 0.$

$$= 3 > 0. \checkmark$$

Case 1:- $u v^i x y^i z \in L, i \geq 0.$

Let, $i = 2$

$$\therefore w = u v^2 x y^2 z$$

$$= a(ab)^2 b c^2 c$$

$$w = aababbbccc \quad \times$$

$$\therefore w \notin L.$$

So, the language is not context-free language. because its not satisfied all cases.